

Bonsai Accelerator Hardware Blueprint

Architecture proposal before implementation - Oriol Pont, July 2026

Scope and Benchmark Basis

This document is the pre-implementation hardware blueprint for the Bonsai accelerator. It describes the architecture, submodules, signal groups, and control FSMs that guide development. It intentionally does not lock down low-level implementation constants such as FIFO depth, exact fixed-point formats, number of compute lanes, or final bus widths, since those should be selected during RTL implementation and synthesis.

The software target is Bonsai-1.7B Q1_0, a compact LLM, as previously discussed. The model image is around 242 MB. Since the course board is a Sipeed Tang Nano 9K with a Gowin LittleBee GW1NR-9 FPGA running a NEORV32 System-on-Chip (SoC), it is far too restricted to hold the full model image.

The three-tier benchmark exploration previously done has allowed defining the scope. To recap, Tier 1 uses `llama.cpp` as the full software reference and shows two important runtime regimes: short-context inference is dominated by Q1_0 matrix operations, while long-context inference shifts toward attention and KV-cache work. Tier 2 exposes the same Bonsai path in a simpler, custom, self-contained C++ runner, which allows measurement of the repeated Q1_0 matrix-vector calls over 128-weight packed groups, plus per-layer attention over growing KV history. Tier 3 then gives reduced cycle-level kernels for the same targets, which can be implemented and measured on the NEORV32 SoC, and that we want to accelerate.

Since Tier 2 is a CPU/software path, it can in principle run on a NEORV32 SoC if enough memory, storage, and simulation time are available. The exact Tier 2 C++/GGUF runner code is host-oriented, but the SoC-facing portable CPU path preserves the same backend boundaries. The fine-grained on-device measurements are restricted to Tier 3 kernels, keeping Tier 2 as the full-model accounting layer used for extrapolation.

The final architecture therefore contains two proposals, each of which addresses one of the two bottlenecks identified:

1. **Proposal A: compute-first tensor accelerator.** accelerate the Q1_0 fixed-weight matrix-vector backend used by Bonsai linear layers.
2. **Proposal B: stream and memory-management improvement.** reduce CPU communication overhead and long-context attention/KV traversal cost with FIFO/stream handshaking and latency hiding, among other possible techniques.

Common Top-Level Architecture

In the proposed architecture, the accelerator is a co-processor to the NEORV32 CPU inside the SoC. The CPU is the system controller, running the Bonsai-like harness that orchestrates model execution and launches accelerator work when reaching the target workloads. As discussed, the **accelerator focuses on providing the demonstration of two services**: a Q1_0 tensor service and a stream-based attention/KV service.

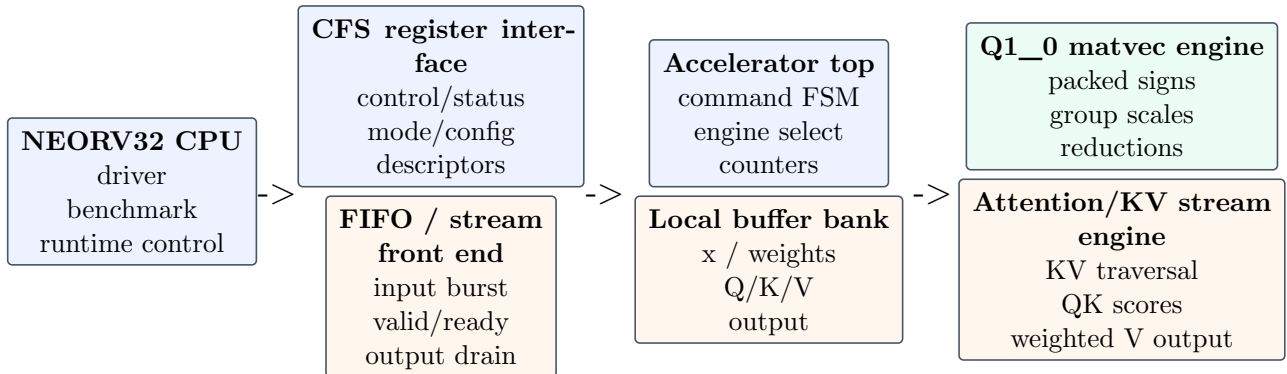


Figure 1: Common architecture used by both proposals. The upper lane is the CFS control path: the CPU configures `accelerator_top`, which runs the command FSM, selects the engine, and exposes counters/status. The lower lane is the data path: bulk payloads enter through the FIFO/stream front end, are staged in local buffers, and are consumed by either the Q1_0 matvec engine or the attention/KV stream engine.

Top-Level Modules and Signal Groups

This section defines the shared accelerator shell before specifying the two engines. The goal is to fix the module boundaries and control flow that both proposals need: CPU-visible control registers, optional FIFO/stream movement, local buffering, engine dispatch, status, and counters. The exact Q1_0 datapath and the exact attention/KV datapath are intentionally left as black-box services.

Top-Level Submodules

Submodule	Responsibility	Stored state
<code>cfs_reg_file</code>	CPU-visible memory-mapped control/status interface. Holds the command descriptor, start/clear bits, mode selection, and readable status/counters.	descriptor fields, status bits
<code>stream_frontend</code>	FIFO/stream data ingress and egress. Accepts burst payloads from the CPU-side interface and drains result words back to software.	FIFO levels, route tags
<code>local_buffer_bank</code>	Shared staging memory between stream input and engines. Stores input vector tiles, packed weight groups, Q/K/V tiles, and output words.	buffer valid bits, read/write pointers
<code>accel_top</code>	Main command controller. Reads the descriptor, checks resources, selects one engine, launches work, waits for completion, and publishes final status.	mode, active engine, command state
<code>q1_matvec_engine</code>	Proposal A service endpoint. Consumes local-buffered vector/weight data and produces matrix-vector results.	engine-local state
<code>attn_kv_engine</code>	Proposal B attention/KV service endpoint. Consumes Q/K/V data through the stream/local-buffer path and produces attention output.	engine-local state
<code>counter_block</code>	Performance and debug counters shared by both proposals: total cycles, CPU-blocked cycles, FIFO stalls, engine-active cycles, and command completions.	cycle and event counters

The top-level shell therefore has two clear boundaries, similarly reflected in the previous diagram. On the CPU side, only `cfs_reg_file` and `stream_frontend` are visible. On the engine side, `accel_top` exposes a common launch/completion contract to either `q1_matvec_engine` or `attn_kv_engine`.

Top-Level Signal Groups

Interface	Direction	Representative signals (TBD)	Purpose
Clock/reset	global in	<code>clk</code> <code>rst_n</code>	Common clock and reset for all accelerator modules.
CFS register bus	CPU <-> CFS	<code>cfs_we</code> <code>cfs_re</code> <code>cfs_addr</code> <code>cfs_wdata</code> <code>cfs_rdata</code>	Memory-mapped CFS access for control, status, descriptor, and counters. Exact address layout remains implementation-defined.
Command descriptor	CFS -> top	<code>cmd_start</code> <code>cmd_mode</code> <code>cmd_clear</code> <code>cmd_error_clear</code> <code>desc_valid</code> <code>desc_shape</code> <code>desc_payload_len</code> <code>desc_result_len</code>	Decoded command and descriptor. <code>cmd_mode</code> selects Q1_0 matvec, attention/KV, or a stream-only diagnostic mode. Width and packing remain implementation-defined.

Interface	Direction	Representative signals (TBD)	Purpose
Status and counters	<code>top/counters</code> -> <code>CFS</code>	<code>status_busy</code> <code>status_done</code> <code>status_error</code> <code>status_engine</code> <code>cnt_enable</code> <code>cnt_event</code> <code>cnt_snapshot</code>	CPU-readable lifecycle status, active/finished engine identity, and performance event capture.
Input stream	<code>CPU</code> -> <code>stream</code>	<code>in_valid</code> <code>in_ready</code> <code>in_data</code> <code>in_last</code>	Input payload stream used by the FIFO/stream front end, especially for Proposal B style transfer.
Output stream	<code>stream</code> -> <code>CPU</code>	<code>out_valid</code> <code>out_ready</code> <code>out_data</code> <code>out_last</code>	Output stream for result words, checksums, or counter snapshots.
Local buffer access	<code>stream/buffer</code> <-> <code>engine</code>	<code>buf_wr_en</code> <code>buf_wr_bank</code> <code>buf_wr_data</code> <code>buf_rd_en</code> <code>buf_rd_bank</code> <code>buf_rd_data</code>	Read/write access to local staging banks shared by the stream front end and selected engine.
Engine command	<code>top</code> <-> <code>engine</code>	<code>engine_start</code> <code>engine_kind</code> <code>engine_busy</code> <code>engine_done</code> <code>engine_error</code>	Common launch/completion handshake. Only one engine is active for a command.
Engine data	<code>engine</code> <-> <code>buffer</code>	<code>engine_in_req</code> <code>engine_in_valid</code> <code>engine_out_valid</code> <code>engine_out_ready</code>	Generic data movement contract between selected engine and local buffers. Engine-specific meaning is defined later.

With these, the high-level design of the accelerator is complete, without prematurely jumping into implementation details. The next section defines the control Finite-State Machines (FSMs) that orchestrate the top-level flow, the stream front end, and the local buffer bank, before moving to the two engines.

Control Finite-State Machines

`accel_top` Finite-State Machine

`accel_top` is the main top-level controller. It validates commands, coordinates buffers/streams, launches the selected engine, and reports completion.

State	Action / outputs	Condition	Next state
RESET	Clear status bits, selected engine, command registers, local control flags, and counter enables.	reset released	IDLE
IDLE	Keep <code>status_busy = 0</code> ; wait for a new command with no uncleared result.	<code>cmd_start</code> <code>desc_valid</code> otherwise	&& LATCH_DESC stay
LATCH_DESC	Copy <code>cmd_mode</code> and descriptor fields from <code>cfs_reg_file</code> into internal command registers.	descriptor latched	CHECK_RESOURCES
CHECK_RESOURCES	Validate selected mode, expected payload/result lengths, and availability of the stream/buffer path.	valid or invalid	PREPARE_INPUT or ERROR

State	Action / outputs	Condition	Next state
PREPARE_INPUT	Set <code>status_busy</code> = 1; request input fill when streamed input is required, or mark input ready when data is already resident.	input ready or stream/buffer fault	START_ENGINE or ERROR
START_ENGINE	Assert <code>engine_start</code> with <code>engine_kind</code> selecting either <code>q1_matvec_engine</code> or <code>attn_kv_engine</code> .	engine accepted or illegal engine response	WAIT_ENGINE or ERROR
WAIT_ENGINE	Keep <code>status_busy</code> = 1; count active/stall events and wait for selected engine completion.	<code>engine_done</code> or <code>engine_error</code>	PUBLISH_RESULT or ERROR
PUBLISH_RESULT	Mark output buffer/FIFO as readable, snapshot counters, and prepare final status fields.	result visible	DONE
DONE	Set <code>status_done</code> = 1, clear <code>status_busy</code> , and hold result/status until software acknowledges.	<code>cmd_clear</code> new start before clear	IDLE stay
ERROR	Set <code>status_error</code> = 1, clear <code>status_busy</code> , latch error code, and block new commands.	<code>cmd_error_clear</code> otherwise	IDLE stay

stream_frontend FSM

State	Action / outputs	Condition	Next state
IDLE	Hold stream counters; keep <code>in_ready/out_valid</code> inactive unless a descriptor requests movement.	input phase or output phase otherwise	ACCEPT_INPUT or SERVE_OUTPUT stay
ACCEPT_INPUT	Assert <code>in_ready</code> when target space exists; accept <code>in_data</code> on <code>in_valid</code> && <code>in_ready</code> .	word accepted or no space or length/packet fault	ROUTE_INPUT or WAIT_SPACE or STREAM_ERROR
ROUTE_INPUT	Use descriptor context to route the accepted word into the selected local-buffer bank and increment payload count.	more input expected or payload complete or buffer unavailable	ACCEPT_INPUT or FLUSH or WAIT_SPACE
WAIT_SPACE	Un-assert <code>in_ready</code> while the target FIFO or local-buffer bank cannot accept another word.	space available or protocol fault	ACCEPT_INPUT or ROUTE_INPUT or STREAM_ERROR
SERVE_OUTPUT	Read result words from output buffer/FIFO and assert <code>out_valid</code> while data is available.	<code>out_valid</code> && <code>out_ready</code> , more data or last result word or underflow	stay or FLUSH or STREAM_ERROR
FLUSH	Close packet boundary, check <code>in_last/out_last</code> , and finalize payload/result counters.	packet consistent or length/last mismatch	STREAM_DONE or STREAM_ERROR
STREAM_DONE	Expose stream completion to <code>accel_top</code> and hold completion until acknowledged.	top acknowledges	IDLE

State	Action / outputs	Condition	Next state
STREAM_ERROR	Latch overflow, underflow, unexpected last, or length mismatch and expose fault to <code>accel_top</code> .	error cleared	IDLE

local_buffer_bank Control FSM

The local buffers are a shared staging area, and specifically not a third accelerator. They decouple CPU/stream timing from engine timing and give both engines a common data source/sink.

State	Action / outputs	Condition	Next state
IDLE	No active ownership change, valid bits and pointers retain their current values.	fill request or engine read request or clear request	FILL_BANK or SERVE_ENGINE or CLEAR_BANK
FILL_BANK	Accept writes from <code>stream_frontend</code> into the selected bank and advance write pointer.	bank filled/payload complete or overflow/conflict	MARK_READY or CLEAR_BANK or error to top
MARK_READY	Set bank valid bit and expose input readiness to <code>accel_top</code> and the selected engine.	read granted or clear request	SERVE_ENGINE or CLEAR_BANK
SERVE_ENGINE	Grant read access to the selected engine and advance read pointer as the engine consumes data.	engine releases input bank or read conflict	CAPTURE_OUTPUT or IDLE or error to top
CAPTURE_OUTPUT	Accept result words from the selected engine into output bank/FIFO and advance output pointer.	result complete or output full	DRAIN_OUTPUT or stay or error to top
DRAIN_OUTPUT	Allow <code>stream_frontend</code> or CFS read path to drain result words from the output bank.	result drained or drain stalled	CLEAR_BANK or stay
CLEAR_BANK	Clear selected valid flags and reset bank pointers for the next command.	clear complete	IDLE

Counter Policy

In terms of counters, the CPU should be able to snapshot the following events for performance analysis:

- `total_cycles`: cycles from accepted `cmd_start` to `DONE` or `ERROR`.
- `engine_active_cycles`: cycles where the selected engine is busy.
- `fifo_wait_cycles`: cycles where input or output transfer is blocked by valid/ready backpressure.
- `buffer_wait_cycles`: cycles where an engine waits for local-buffer data or output space.
- `cpu_blocked_cycles`: cycles measured for blocking software interaction, in Proposal B evaluation.

This completes the top-level blueprint at the SoC level and its contract and control interface. Of course, this is to guide the design process, and can be refined as the design progresses. The next sections specialize the two rightmost services separately: first the Q1_0 matvec engine for Proposal A, then the attention/KV stream engine and FIFO/memory-management candidate choices for Proposal B.

Proposal A: q1_matvec_engine

Proposal A is the compute-first tensor accelerator. The target primitive is Q1_0 matrix-vector multiplication over fixed-weight rows, because this is the repeated backend used by Bonsai linear layers. Decode naturally uses one matrix-vector product per layer operation for the current token. Prefill uses the same linear transforms repeated over prompt positions or batches, so this engine is a suitable board-sized primitive for extrapolating Bonsai impact from Tier 3 measurements.

The engine consumes local-buffered activation tiles and packed Q1_0 weight groups. Each group contains sign bits and a scale. The hardware performs sign-controlled add/sub reductions, applies the scale, accumulates the row result, and writes one output element.

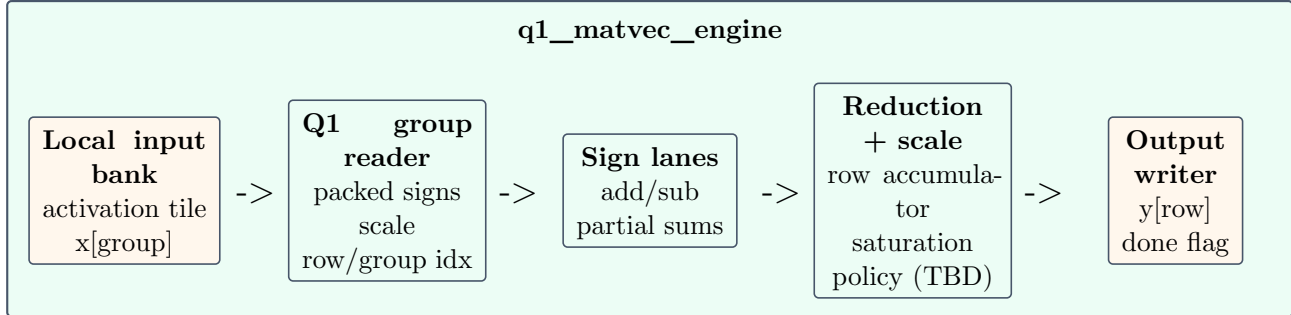


Figure 2: Zoom-in block diagram for q1_matvec_engine.

Q1_0 Matvec FSM

This FSM is local to q1_matvec_engine. It assumes accel_top already selected the engine and that local buffers expose the activation and packed-weight payloads through the shared engine/data contract.

State	Action / outputs	Condition	Next state
IDLE	Keep engine_busy = 0; wait for the top-level launch handshake.	engine_start engine_kind q1_matvec	with = LOAD_DESC
LOAD_DESC	Latch rows, columns/group count, buffer selectors, and output mode from the command descriptor.	descriptor accepted or descriptor invalid	LOAD_X or ERROR
LOAD_X	Request or select the activation tile from local_buffer_bank.	activation tile ready or buffer fault	INIT_ROW or ERROR
INIT_ROW	Clear row accumulator and initialize group_idx for the current output row.	row initialized	READ_GROUP
READ_GROUP	Read packed signs and scale for the current row/group.	group word valid or input unavailable	DOT_GROUP or WAIT_INPUT
WAIT_INPUT	Hold row/group counters while waiting for buffer data.	group word valid or timeout/protocol fault	DOT_GROUP or ERROR
DOT_GROUP	Use sign bits to select add/sub operations over the activation group and produce a group partial sum.	partial sum ready	ACCUM_ROW
ACCUM_ROW	Apply group scale and accumulate into the row accumulator.	more groups or row complete	READ_GROUP or WRITE_ROW
WRITE_ROW	Write the completed row result to the output buffer path.	output accepted or output blocked	NEXT_ROW or WAIT_OUTPUT

State	Action / outputs	Condition	Next state
WAIT_OUTPUT	Hold row result until output space is available.	output accepted or protocol fault	NEXT_ROW or ERROR
NEXT_ROW	Advance row counter and clear per-row state when more output rows remain.	more rows or all rows complete	INIT_ROW or DONE
DONE	Assert <code>engine_done</code> , clear <code>engine_busy</code> , and expose engine-local counters.	top acknowledges	IDLE
ERROR	Assert <code>engine_error</code> , clear <code>engine_busy</code> , and latch the local failure reason.	top clears command	IDLE

Proposal B: `attn_kv_engine`

Proposal B is the stream and memory-management path. The attention/KV engine is framed this way because long-context cost is dominated by repeatedly moving and traversing K/V history, even though the service can still implement full attention semantics.

The intended service loads Q for a head, streams K over the context window to compute QK scores, normalizes the scores with a later-chosen softmax or online-softmax approximation or implementation, streams V, accumulates the weighted output, and writes the attention result.

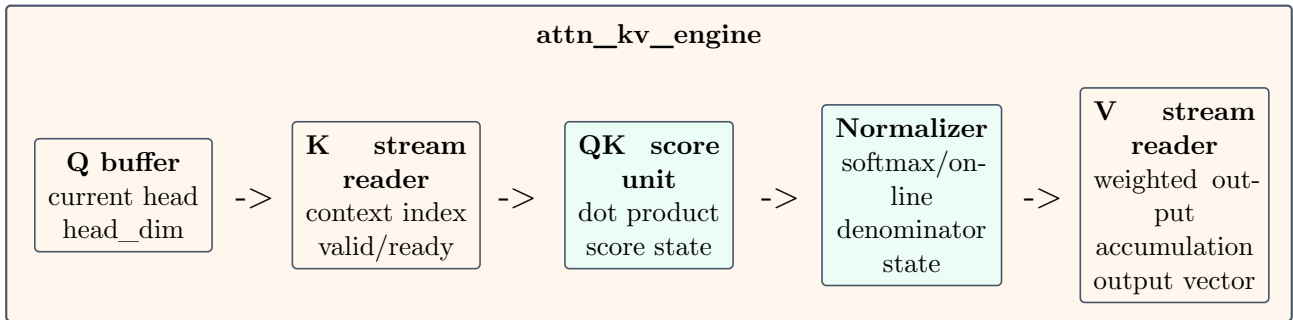


Figure 3: Zoom-in block diagram for `attn_kv_engine`.

Attention/KV Engine FSM

It shares the same engine launch/completion contract as the Q1_0 engine, but its internal phases are organized around Q loading, K traversal, score normalization, V traversal, and output writeback.

State	Action / outputs	Condition	Next state
IDLE	Keep <code>engine_busy</code> = 0; wait for the top-level launch handshake.	<code>engine_start</code> with <code>engine_kind</code> = <code>attn_kv</code>	LOAD_DESC
LOAD_DESC	Latch head count, KV-head mapping, head dimension, context length, buffer selectors, and output mode.	descriptor accepted or descriptor invalid	LOAD_Q or ERROR
LOAD_Q	Load or select Q data for the current head from the local buffer path.	Q ready or buffer fault	INIT_HEAD or ERROR
INIT_HEAD	Clear score/normalization state, output accumulator, and context counters for the current head.	head initialized	SCAN_K
SCAN_K	Stream K data over the context window and compute QK scores.	more K positions or all K scored or stream fault	SCAN_K or NORMALIZE or ERROR

State	Action / outputs	Condition	Next state
NORMALIZE	Normalize scores using the selected softmax or online-normalization method.	normalization complete or numeric/protocol fault	SCAN_V or ERROR
SCAN_V	Stream V data paired with normalized scores and accumulate the weighted output vector.	more V positions or output complete or stream fault	SCAN_V or WRITE_OUT or ERROR
WRITE_OUT	Write the completed attention output vector to the output buffer path.	output accepted or output blocked	NEXT_HEAD or WAIT_OUTPUT
WAIT_OUTPUT	Hold output data until the output buffer/FIFO can accept it.	output accepted or protocol fault	NEXT_HEAD or ERROR
NEXT_HEAD	Advance head counter and select the next Q/K/V slice when more heads remain.	more heads or all heads complete	LOAD_Q or DONE
DONE	Assert <code>engine_done</code> , clear <code>engine_busy</code> , and expose attention/KV counters.	top acknowledges	IDLE
ERROR	Assert <code>engine_error</code> , clear <code>engine_busy</code> , and latch the local failure reason.	top clears command	IDLE

Engine-Level Evaluation Counters

Both engine FSMs should feed the shared `counter_block`, but the report should keep their counters conceptually separate:

- `q1_group_cycles` and `q1_output_rows`: compute work completed by the Q1_0 engine.
- `q1_wait_input_cycles` and `q1_wait_output_cycles`: stalls caused by local-buffer or stream availability.
- `attn_k_positions` and `attn_v_positions`: streamed context work completed by the attention/KV engine.
- `attn_score_cycles`, `attn_norm_cycles`, and `attn_value_cycles`: phase-level attention timing, for extrapolation.
- `attn_wait_stream_cycles`: cycles where attention is ready but the KV stream or output path is not.

Summary

This document has defined the first iteration of the top-level architecture, submodules, signal groups, and control FSMs for the proposed Bonsai accelerator. We want to focus on gaining performance in both the Q1_0 matvec backend and the attention/KV long-context path, measuring with precision in simulation several set-ups, to then be able to extrapolate the impact of the accelerator on Bonsai-1.7B Q1_0 inference, which was the original motivation for this work despite the constraints of the course board. The next step is to implement the RTL for the top-level shell and both engines starting from this architecture, to validate if any assumptions need to be revised, or if the proposed FSMs and signal groups are sufficient to support this initially intended services.